

M1 Conversion Core Design

October 9, 2026

Contents

1. Summary
 2. Background and goal
 3. Design
 4. Implementation plan
 5. Evaluation
 6. Risks
- References

Purpose. This document describes the data structures, calculation paths, error rules and test method of the conversion function `convert(value, src, dst)` to be built in M1, and seeks approval of the method and the completion criteria before implementation starts.

1 Summary

M1 builds one conversion function that can be called from Python without a command line, together with its tests. M2 only wraps this function[D1].

- **Structure.** `convert.py` holds two factor dictionaries for length and weight, one dictionary of temperature conversion formulas, and one function, `convert()`. It has no imports.
- **Rules.** Unit names are case-insensitive. An unknown unit, units of different kinds, and a temperature below 0 K raise `ValueError`.
- **Tests.** `test_convert.py` (unittest) checks 9 defined values, 5 temperature reference points, the round trip of 109 same-kind unit pairs, and 3 error types. Comparison uses a relative error within $1e-12$.

2 Background and goal

The proposal was approved on 2026-10-09[D1]. The repository has no code yet[D2]. Of the proposal's success criteria (Table 4), M1 covers accuracy, round-trip consistency, dependencies, and three of the four error-handling cases. The remaining case (non-numeric value) arises in the step that turns a string into a number, so M2 covers it. Table 1 shows this split.

Table 1. Proposal success criteria and M1's share. [D1]

Proposal criterion	M1	M2
Accuracy	Defined-value and reference-point tests	—
Round-trip consistency	Tests on all unit pairs	—
Error handling	Unknown unit, kind mismatch, below 0 K	Non-numeric value, exit code and message
Dependencies	No imports in <code>convert.py</code>	Adds only <code>argparse</code> , <code>sys</code>

3 Design

Figure 1 shows the flow inside `convert()`. It looks up the unit names, checks that the two units are of the same kind, and then calculates along one of two paths depending on the kind. The dashed lines below mark where it exits with an error.

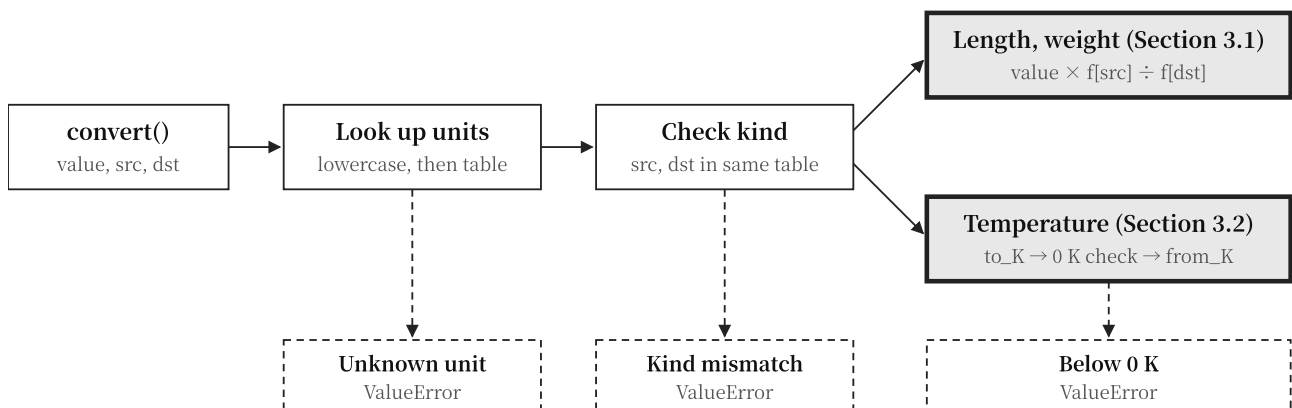


Figure 1. Flow inside `convert()`. Bold borders mark the two paths where the actual calculation happens, and dashed lines mark the exits to errors. All parts are newly built in this milestone.

3.1 Length and weight: factor tables

For each unit, a dictionary holds the value of one unit in the base unit (the factor). Conversion consists only of **multiplying by the source unit's factor to get the base unit, then dividing by the target unit's factor**. For example, $100 \text{ km} \rightarrow \text{mi}$ is $100 \times 1000 \div 1609.344 = 62.137\dots$. The factors are copied directly from the defined values in Table 3 of the proposal[D1, R1, R2]. Table 2 shows the three data structures.

Table 2. Data structures in convert.py. f is the value of one unit in the base unit. [D1, R1, R2]

Name	Form	Contents
LENGTH	unit $\rightarrow$ f (m)	mm 0.001, cm 0.01, m 1, km 1000, in 0.0254, ft 0.3048, yd 0.9144, mi 1609.344
MASS	unit $\rightarrow$ f (kg)	mg 0.000001, g 0.001, kg 1, t 1000, lb 0.45359237, oz 0.45359237 / 16
TEMP	unit $\rightarrow$ (to kelvin, from kelvin)	c: (t + 273.15, k - 273.15), f: ((t + 459.67) $\div$ 1.8, k $\times$ 1.8 - 459.67), k: (unchanged, unchanged)

3.2 Temperature: formulas via kelvin

Temperature cannot be converted by multiplication alone, so each unit has a pair of formulas: “to kelvin” and “from kelvin”. Conversion applies the source unit's first formula to get kelvin, then the target unit's second formula. With n units there are n formula pairs, so pairwise formulas such as $^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F}$ are not written separately. The constants in the formulas (32, 1.8, 459.67, 273.15) are NIST values[R1]. **A kelvin value below 0 is a physically impossible temperature, so it raises an error.** 0 K itself (-273.15°C , -459.67°F) is allowed.

3.3 Unit name rules

Input unit names are lowercased before lookup, so KM , Km and km are all the same. We checked that the 17 table keys do not collide when lowercased (Table 2). Unit names are accepted as symbols only. Long names such as `meter` or `celsius` are not accepted in this milestone. `t` is the metric tonne (1000 kg).

3.4 Errors

All errors are raised as a single `ValueError`, with the reason in the message. Table 3 lists the three cases and example messages. M2 only needs to catch this exception, print the message and exit with exit code 2. Converting a unit to itself ($\text{km} \rightarrow \text{km}$) is not an error; it returns the value unchanged.

Table 3. Error cases and example messages.

Case	Example input	Example message
Unknown unit	<code>convert(1, "km", "xx")</code>	unknown unit 'xx' (known: mm, cm, m, km, ...)
Kind mismatch	<code>convert(1, "km", "kg")</code>	cannot convert length to mass
Below 0 K	<code>convert(-300, "c", "f")</code>	-300 c is below absolute zero

4 Implementation plan

There are two files and no new dependencies. `convert.py` does not import even the standard library. Table 4 lists the changes per file.

Table 4. Files changed.

File	Change	Reason
<code>convert.py</code> (new file)	LENGTH, MASS, TEMP, <code>convert()</code>	Sections 3.1–3.4
<code>test_convert.py</code> (new file)	4 groups of <code>unittest</code> tests (Section 5, Table 5)	Check completion criteria

The work order is as follows.

- ① **Tests first.** Write the defined-value and reference-point tests first and confirm that they fail.
- ② **Tables and function.** Write the data in Table 2 and `convert()` so that ① passes.
- ③ **Round-trip and error tests.** Add the remaining tests and run them all with `python -m unittest`.

5 Evaluation

Floating-point arithmetic does not guarantee exact decimal results. Calculating the designed formulas in advance showed differences in the last digit, as in Table 6[D3]. Therefore **the tests compare with `math.isclose(rel_tol=1e-12, abs_tol=1e-12)` instead of equality**[R3]. When the expected value is 0 (32 °F → °C), relative error is undefined, so an absolute error of 1e-12 is also set. Table 5 lists the test groups and completion criteria.

Table 5. Test groups and completion criteria. The number of round-trip pairs is length 8×8 + weight 6×6 + temperature $3 \times 3 = 109$.

Test group	Contents	Target
Defined values	1 in = 0.0254 m, 1 ft, 1 yd, 1 mi, 1 km, 1 lb, 1 oz = 1 lb $\div$ 16, 1 t, 1 mg	9 / 9 pass
Temperature reference points	-40 C = -40 F, 100 C = 212 F, 0 C = 273.15 K, 32 F = 0 C, 0 K = -459.67 F	5 / 5 pass
Round trip	A $\rightarrow$ B $\rightarrow$ A for every same-kind pair, value 25 (at or above 0 K on all three temperature scales)	109 / 109 pass
Errors and name rules	The 3 cases in Table 3 raise <code>ValueError</code> ; KM $\rightarrow$ MI equals km $\rightarrow$ mi	4 / 4 pass
Dependencies	import statements in <code>convert.py</code>	0

Table 6. Actual computed values of the designed formulas. Measured on Python 3.11.16. [D3]

Formula	Computed	Exact	Relative error
$(100 + 273.15) \times 1.8 - 459.67$	211.99999999999994	212	2.7e-16
$(-40 + 273.15) \times 1.8 - 459.67$	-40.000000000000006	-40	1.5e-15
$0.0254 \div 0.01$	2.54	2.54	0
$(32 + 459.67) \div 1.8 - 273.15$	0.0	0	0

6 Risks

Table 7 lists problems that can arise in M1 and where each one shows up.

Table 7. Risks and how to detect them.

Risk	How it is detected	Response
Typo in a factor	The defined-value tests fail, because they hold their expected values separately.	Factors are copied as in Table 2; test expected values are copied separately from Table 3 of the proposal.
Accumulated error exceeds $1\text{e-}12$	Round-trip tests fail. The largest error in Table 6 is $1.5\text{e-}15$, a margin of about 670 times.	If exceeded, record the cause formula in the result document and adjust the tolerance after approval.
Rounding gives a negative value at the 0 K boundary	Shows up if the 0 K = -459.67 F test or a -273.15 C input exits with an error.	In Table 6 the boundary value computed to exactly 0.0. If it deviates, report it in the result document.

References

- [R1] NIST. NIST Guide to the SI, Appendix B.8: Factors for Units Listed Alphabetically (SP 811).
<https://www.nist.gov/pml/special-publication-811/nist-guide-si-appendix-b-conversion-factors/nist-guide-si-appendix-b8> (accessed 2026-10-09)
- [R2] Wikipedia. International yard and pound.
https://en.wikipedia.org/wiki/International_yard_and_pound (secondary source; the original 1959 agreement was not verified)
- [R3] Python Software Foundation. math.isclose.
<https://docs.python.org/3/library/math.html#math.isclose>
- [D1] Unit Conversion CLI Proposal (docs/reports/unit-converter-cli/00-proposal.pdf), approved 2026-10-09.
- [D2] Repository initial commit 254d145 “init” (no files).
- [D3] Calculation check of the designed formulas, Python 3.11.16, 2026-10-09.